

Agent Phantom Recovery

13-api-specification.md

Production API Specification

Version: 1.0

Part 3

27. Tool APIs

Purpose

Tool APIs provide a unified execution interface for all platform tools.

The frontend should never communicate directly with individual tool implementations.

Instead:

```
Antigravity IDE
  ↓
Tool API
  ↓
Tool Service
  ↓
Tool Adapter
  ↓
Actual Tool
```

This allows tools to be replaced without affecting the frontend.

28. Tool Registry APIs

List Available Tools

Endpoint

```
GET /api/v1/tools
```

Response

```
{
  "tools": [
    {
      "name": "terminal",
      "status": "AVAILABLE"
    }
  ]
}
```

Get Tool Details

Endpoint

```
GET /api/v1/tools/{toolName}
```

Response

```
{
  "name": "terminal",
  "description": "",
  "capabilities": []
}
```

Tool Health

Endpoint

```
GET /api/v1/tools/health
```

29. Tool Execution APIs

Execute Tool

Endpoint

```
POST /api/v1/tools/execute
```

Request

```
{
  "task_id": "",
  "tool": "terminal",
  "action": "run",
  "parameters": {}
}
```

Response

```
{
  "execution_id": "",
  "status": "QUEUED"
}
```

Execution Status

Endpoint

```
GET /api/v1/tools/executions/{executionId}
```

Cancel Tool Execution

Endpoint

```
POST /api/v1/tools/executions/{executionId}/cancel
```

30. Browser APIs

Purpose

These APIs power the Browser Workspace inside Antigravity.

The browser is not merely a webpage preview.

It is an agent-controlled execution surface.

Create Browser Session

Endpoint

```
POST /api/v1/browser/sessions
```

Request

```
{  
  "task_id": ""  
}
```

Response

```
{  
  "session_id": ""  
}
```

Navigate Browser

Endpoint

```
POST /api/v1/browser/sessions/{sessionId}/navigate
```

Request

```
{  
  "url": "https://example.com"  
}
```

Refresh Page

Endpoint

```
POST /api/v1/browser/sessions/{sessionId}/refresh
```

Browser State

Endpoint

```
GET /api/v1/browser/sessions/{sessionId}/state
```

Response

```
{
  "url": "",
  "title": "",
  "status": "READY"
}
```

Capture Screenshot

Endpoint

```
POST /api/v1/browser/sessions/{sessionId}/screenshot
```

Response

```
{
  "artifact_id": ""
}
```

Browser DOM Snapshot

Endpoint

```
GET /api/v1/browser/sessions/{sessionId}/dom
```

Browser Observations

Endpoint

```
GET /api/v1/browser/sessions/{sessionId}/observations
```

Browser Action History

Endpoint

```
GET /api/v1/browser/sessions/{sessionId}/history
```

31. Browser Automation APIs

Execute Browser Action

Endpoint

```
POST /api/v1/browser/sessions/{sessionId}/action
```

Request

```
{  
  "action": "click",  
  "selector": "#submit"  
}
```

Supported Actions

```
CLICK  
TYPE  
SCROLL  
SELECT  
HOVER  
WAIT  
UPLOAD  
DOWNLOAD
```

Multi-Step Browser Workflow

Endpoint

```
POST /api/v1/browser/sessions/{sessionId}/workflow
```

Purpose

Allows agents to perform structured browser tasks.

Example:

```
Open Login Page
↓
Enter Credentials
↓
Submit
↓
Inspect Dashboard
```

32. Terminal APIs

Purpose

Provide persistent terminal execution.

Terminal state must survive page refreshes.

Create Terminal Session

Endpoint

```
POST /api/v1/terminal/sessions
```

Session Details

Endpoint

```
GET /api/v1/terminal/sessions/{sessionId}
```

Execute Command

Endpoint

```
POST /api/v1/terminal/sessions/{sessionId}/execute
```

Request

```
{  
  "command": "pytest"  
}
```

Response

```
{  
  "execution_id": ""  
}
```

Command History

Endpoint

```
GET /api/v1/terminal/sessions/{sessionId}/history
```

Kill Process

Endpoint

```
POST /api/v1/terminal/processes/{processId}/kill
```

Terminal Environment Info

Endpoint

```
GET /api/v1/terminal/sessions/{sessionId}/environment
```


33. File System APIs

Purpose

Provide repository interaction.

These APIs power the Source Code Workspace.

Repository File Tree

Endpoint

```
GET /api/v1/files/tree
```

Read File

Endpoint

```
GET /api/v1/files/read
```

Query

```
?path=
```

Write File

Endpoint

```
POST /api/v1/files/write
```

Request

```
{
  "path": "",
  "content": ""
}
```

Delete File

Endpoint

```
DELETE /api/v1/files/delete
```

Move File

Endpoint

```
POST /api/v1/files/move
```

Search Files

Endpoint

```
POST /api/v1/files/search
```

Request

```
{
  "query": ""
}
```

34. Repository Intelligence APIs

Purpose

Expose repository analysis capabilities.

Symbol Search

Endpoint

```
GET /api/v1/repositories/{repositoryId}/symbols
```

Find References

Endpoint

```
GET /api/v1/repositories/{repositoryId}/references
```

Call Graph

Endpoint

```
GET /api/v1/repositories/{repositoryId}/call-graph
```

Dependency Graph

Endpoint

```
GET /api/v1/repositories/{repositoryId}/dependency-graph
```

Architecture Summary

Endpoint

```
GET /api/v1/repositories/{repositoryId}/architecture
```

Semantic Code Search

Endpoint

```
POST /api/v1/repositories/{repositoryId}/semantic-search
```

Request

```
{
  "query": "authentication middleware"
}
```

35. Git APIs

Purpose

Git is a first-class component.

Agent outputs should be traceable through Git operations.

Repository Status

Endpoint

```
GET /api/v1/git/status
```

Branch List

Endpoint

```
GET /api/v1/git/branches
```

Create Branch

Endpoint

```
POST /api/v1/git/branches
```

Commit Changes

Endpoint

```
POST /api/v1/git/commit
```

Request

```
{
  "message": ""
}
```

Diff

Endpoint

```
GET /api/v1/git/diff
```

Staged Changes

Endpoint

```
GET /api/v1/git/staged
```

36. GitHub APIs

Purpose

Support repository collaboration.

Repository Metadata

Endpoint

```
GET /api/v1/github/repository
```

Create Pull Request

Endpoint

```
POST /api/v1/github/pull-requests
```

Request

```
{
  "title": "",
  "description": ""
}
```

Pull Request Status

Endpoint

```
GET /api/v1/github/pull-requests/{prId}
```

Pull Request Review

Endpoint

```
POST /api/v1/github/pull-requests/{prId}/review
```

37. Execution APIs

Purpose

Provide visibility into autonomous execution.

These APIs power Antigravity's execution timeline.

Start Execution

Endpoint

```
POST /api/v1/executions
```

Execution Details

Endpoint

```
GET /api/v1/executions/{executionId}
```

Active Executions

Endpoint

```
GET /api/v1/executions/active
```

Pause Execution

Endpoint

```
POST /api/v1/executions/{executionId}/pause
```

Resume Execution

Endpoint

```
POST /api/v1/executions/{executionId}/resume
```

Cancel Execution

Endpoint

```
POST /api/v1/executions/{executionId}/cancel
```

38. Execution Timeline APIs

Timeline Events

Endpoint

```
GET /api/v1/executions/{executionId}/timeline
```

Execution Logs

Endpoint

```
GET /api/v1/executions/{executionId}/logs
```

Execution Checkpoints

Endpoint

```
GET /api/v1/executions/{executionId}/checkpoints
```

39. Report APIs

Purpose

Generate deliverables.

Generate Report

Endpoint

```
POST /api/v1/reports/generate
```

Report Status

Endpoint

```
GET /api/v1/reports/{reportId}
```

Download Report

Endpoint

```
GET /api/v1/reports/{reportId}/download
```

List Reports

Endpoint

```
GET /api/v1/reports
```

40. Artifact APIs

Artifacts represent generated outputs.

Examples:

- screenshots
 - patches
 - exports
 - logs
 - archives
-

Upload Artifact

Endpoint

```
POST /api/v1/artifacts
```

Get Artifact

Endpoint

```
GET /api/v1/artifacts/{artifactId}
```

Download Artifact

Endpoint

```
GET /api/v1/artifacts/{artifactId}/download
```

Delete Artifact

Endpoint

```
DELETE /api/v1/artifacts/{artifactId}
```

41. Antigravity Workspace Mapping

Browser Workspace

Powered by:

```
/browser/*
```

Capabilities:

- browsing
 - screenshots
 - automation
 - observations
-

Terminal Workspace

Powered by:

```
/terminal/*
```

Capabilities:

- commands
 - processes
 - logs
 - environments
-

Source Code Workspace

Powered by:

```
/files/*  
/repositories/*
```

Capabilities:

- editing
 - search
 - architecture analysis
-

Git Workspace

Powered by:

```
/git/*  
/github/*
```

Capabilities:

- commits
 - branches
 - pull requests
-

Timeline Workspace

Powered by:

```
/executions/*
```

Capabilities:

- progress tracking
 - event inspection
 - checkpoint recovery
-

42. Internal Service Contracts

Tool Execution Result

```
{  
  "execution_id": "",  
  "success": true,
```

```
"output": {},
"metadata": {}
}
```

Browser Result

```
{
  "session_id": "",
  "url": "",
  "observations": []
}
```

Terminal Result

```
{
  "command": "",
  "exit_code": 0,
  "output": ""
}
```

Git Result

```
{
  "commit_id": "",
  "status": "SUCCESS"
}
```

End of API Specification Part 3

Next Section:

Part 4

- Realtime APIs
- WebSocket Protocol
- Event APIs
- Monitoring APIs
- Admin APIs
- Error Specification
- Rate Limiting

- Internal Service Communication
- Production Security Requirements
- Future API Extensions

This final section will define how Antigravity maintains real-time synchronization between the browser workspace, source code workspace, terminal workspace, memory system, planner, reasoner, verifier, and execution timeline.